

Networks degrade for weeks before anyone calls

Telemetry

Events

17 September 2026 · 8 minute read

In most organisations, the network finds out something is wrong when a user calls. By then the event is over and all that is left is explaining it. This article is about the difference between a snapshot and a time series, and how to write event rules that do not end up silenced.

You find out when the user calls

Ask around your organisation how you learn that something is wrong on the network. The honest answer, in most cases, is that someone calls. A user at the south site reports the application is slow, and that is where the work starts: check interfaces, look at utilisation, search logs, compare against last week.

By the time the network team confirms there was saturation on the link, the event is over. It can be explained. It could not be prevented, and you probably do not know how long it had been degrading.

The underlying problem is that an audit — even one covering configuration and state — is a *snapshot*. It answers what is happening now, very well. It does not answer since when, with what trend, or whether this had happened before.

An audit tells you how the network is. A time series tells you where it is heading.

What measuring over time adds

Storing metrics as time series — processor, memory, temperature, utilisation, and errors per interface — changes the kind of question you can answer:

- **Since when.** That device's processor is not high as of today: it rose eleven days ago and never came back down. That points to a change, not a spike.
- **With what trend.** The wide area link is at seventy per cent and growing five points a quarter. The budget conversation stops being an opinion.
- **Whether it is normal.** Eighty per cent utilisation on a Monday at nine may be the usual behaviour. Without history there is no way to know, and you chase false alarms.
- **What happened during the incident.** The window has closed, but the counters remain. It can be reconstructed instead of relying on what someone remembers.

There is an important practical consequence: this has a different consumption profile from the rest of the platform. Storing time series grows with the number of devices *multiplied* by the sampling frequency, and that is an infrastructure and cost decision, not a checkbox you tick.

From measuring to reacting

Having the metric and watching it on a dashboard is still reactive operation, just with better information. The next leap is for the platform to react on its own to defined conditions.

An event rule is simple to state: a condition and an action. The hard part is not writing it, it is scoping both halves well.

The condition has to be specific enough not to over-fire. "High processor" generates noise. "Processor above threshold for more than ten sustained minutes on a distribution device" is actionable. A rule that fires often gets silenced, and a silenced rule is worse than no rule: it creates the feeling of coverage without providing it.

The action has to be proportional to the risk. Collecting diagnostics, opening a record, or notifying is safe and can be automated without hesitation. Changing configuration automatically in response to an event is another category: it is exactly the scenario described in [the article on closing the loop](#), now with nobody watching and at three in the morning.

Our position is that most rules should collect evidence and notify, not modify. The value is that by the time the engineer opens the case, the diagnosis is already done.

Which events are worth watching

The ones that indicate lost redundancy or progressive degradation, which are precisely the ones users do not report yet:

- **Border gateway session down.** Rarely cosmetic: it almost always means a path disappeared.
- **Interior protocol neighbour down.** The network reconverges and keeps working, which is why nobody notices an alternate path was lost. The next incident has nothing left to fall back on.
- **Interface down on a redundant link.** The service stays up. The redundancy does not.
- **Processor, memory, and temperature thresholds.** All three warn before failing, if someone is looking.
- **Rising errors on an interface.** A degrading transceiver or cable gives signals for weeks.

They all share something: **none of them produces a user call.** They are exactly the ones only detected if something is watching for them.

How Oktavia handles it

Telemetry and event orchestration are licensed separately from the rest of Oktavia, for the reason explained above: they store time series and run a permanent evaluator, so they have a different

consumption profile. Not every network needs them on day one, and those that do can add them whenever they choose.

What they include: real-time processor, memory, and interface metrics as time series, with thresholds and trends; native dashboards and Grafana compatibility, so metrics can also be consumed from the tools the organisation already uses; and the event automation engine, with condition-to-action rules covering border gateway session drops, interior protocol neighbour drops, interfaces down, and processor, memory, temperature, and error thresholds.

All of it over the same inventory and the same source of truth as the rest of the platform. An event knows which device it belongs to, which site it is at, what role it plays, and what its intended configuration is, without anyone maintaining a second device list for the monitoring system.

How do you find out something is wrong today?

If the answer is "when they call", there is a window between degradation and report that is currently uncovered. To quote, all we need is roughly how many devices you manage.

About Oktanet

Oktanet is a Mexican network automation company focused on controlled, traceable, and scalable operations for enterprise environments across Latin America.

[LinkedIn](#) · [YouTube](#)

© 2026 Oktanet · Oktavia · Public article, with no confidential customer information.